SCIENCES SORBONNE UNIVERSITÉ

UFR d'Ingénierie

3 décembre 2025

L2 Informatique (2025-2026)

Logique (UL2IN007)

TP

Thème 10

Thème 10 - TP

Objectifs

- Programmation prolog

Quelques rappels de syntaxe prolog

Eléments de base Prolog est un langage de programmation fondé sur la logique des prédicats du premier ordre.
- Les constantes sont représentées par des chaînes de caractères qui commencent par une minuscule.
- Les variables sont représentées par des chaînes de caractères qui commencent par une Majuscule.
- Les prédicats sont représentés par des chaînes de caractères qui commencent par une minuscule.

Ainsi, dans la proposition `frere(X,charlemagne)`, le prédicat est `frere`, d'arité 2 : il est appliqué à la variable `X` et à la constante `charlemagne`.

Les règles sont de la forme `prop :- prop1, prop2, prop3.` qui correspond à la formule de LPPO $(prop1 \wedge prop2 \wedge prop3) \rightarrow prop$.

Le prédicat à gauche de `:-` est la *tête* de la règle et les propositions à droite définissent le *corps* de la règle. Les propositions du corps sont séparées par des virgules.

Prolog repose sur la représentation clausale des formules : la formule
$((prop_1 \wedge prop_2 \wedge prop_3) \vee (prop_4 \wedge prop_5)) \rightarrow prop$, dont la forme normale conjonctive est
$(\neg prop_1 \vee \neg prop_2 \vee \neg prop_3 \vee prop) \wedge (\neg prop_4 \vee \neg prop_5 \vee prop)$, est représentée par les 2 règles

```
prop :- prop1, prop2, prop3.
prop :- prop4, prop5.
```

Chaque règle correspond à une clause comportant au plus un littéral positif.

Les faits correspondent aux clauses qui contiennent un littéral positif et aucun littéral négatif. Ils sont représentés par des règles dont le corps est vide, c'est-à-dire de la forme `prop.`

Programme Les faits et les règles sont écrits dans un fichier texte, dont le nom a pour extension `pl`. Il est préférable que le nom du fichier commence par une minuscule (cf ci-dessous la procédure de chargement du fichier).

Tout éditeur de texte peut être utilisé (par défaut, les fichiers d'extension `pl` sont considérés comme du `perl`, une coloration syntaxique pour prolog peut néanmoins être spécifiée. Par exemple avec `emacs`, on y accède avec alt+x prolog-mode).

Attention : il faut regrouper dans le fichier tous les faits et règles ayant la même tête, sinon l'interprète prolog provoque une erreur lors du chargement du fichier.

Le symbole `%` met en commentaire tout ce qui suit, sur la ligne où il figure.

Les symboles `/* */` permettent de mettre des commentaires plusieurs lignes.

L'interprète `swiprolog` Pour lancer l'interprète prolog, il faut taper dans un terminal la commande `swipl`. On entre en mode "requête", indiqué par l'invite `?-`

Toutes les commandes se terminent par un point. On peut en particulier utiliser
- `[nom].` ou `consult(nom).` ou `['nom.pl'].` : charge le fichier `nom.pl`.

  La commande `[nom].` ne convient pas si le nom du fichier commence par une majuscule : prolog considère que l'on fait référence à une variable et provoque une erreur. Elle ne convient pas non plus si le nom du fichier contient des caractères spécifiques (tiret, point, etc).

  A chaque nouveau chargement d'un fichier existant, les faits et les règles précédents sont écrasés par les nouvelles définitions.

- `help.` : affiche la fenêtre d'aide et la liste des prédicats prédéfinis dans SWI-Prolog.
- `help(pred).` : affiche l'aide concernant le prédicat `pred`.
- `trace.` : permet de suivre pas à pas l'évaluation des requêtes.
- `notrace.` et `nodebug.` : permettent de quitter le mode trace.
- `halt.` : quitte Prolog.

Requêtes Une fois le programme chargé, on peut entrer une requête, c'est-à-dire une proposition de la LPPO contenant des variables écrite avec la syntaxe prolog, qui se termine par "`.`".

Exercices

Exercice 13 – Premières requêtes

Question 1

Créer un fichier `exercice0.pl` contenant les deux faits

```
toto(a,b).
toto(c,b).
```

1. Le charger : l'affichage doit alors être de la forme

   ```
   ?- [exercice0].
   //1 compiled 0.01 sec, 624 bytes
   true.
   ```

2. Tester la requête `?- toto(X,b).` et comprendre les résultats obtenus.

   S'il n'y a qu'une instanciation possible, l'interprète rend la main. Sinon, il attend une saisie de l'utilisateur : la touche "Entrée" termine, la touche "`;`" donne successivement les autres solutions obtenue par exploration de l'arbre de résolution.

3. Même question avec `?- toto(b,X).`

4. Reprendre les deux exemples précédents en mode trace, pour observer les étapes d'unification effectuées

   NB prolog renomme toutes les variables avec des noms de la forme $_G2673$ ou un autre entier.

Question 2

Tester en mode trace les deux programmes du dernier exercice de TD sur l'arbre de résolution au format prolog.

Exercice 14 – Relations familiales

1. Télécharger le fichier `familleSimpson.pl` disponible sur le moodle et le charger dans prolog.

   Les prédicats binaires `pere(X, Y)` et `mere(X, Y)` indiquent que `X` est le père (respectivement la mère) de `Y`.

2. Dessiner sur papier l'arbre généalogique correspondant.

3. Pour chacune des requêtes suivantes, indiquer en français quelle est la question qu'elle représente, prédire les réponses attendues puis vérifier que prolog les donne bien.

   (a) `pere(clancy, X).`

   (b) `mere(X, homer).`

   (c) `mere(homer, X).`

(d) `mere(X, herb).`
(e) `pere(homer, X), feminin(X).`
(f) `feminin(X), pere(homer, X).`

Pour cette requête et la précédente, examiner l'arbre de résolution en utilisant le mode trace et commenter les différences observées.

(g) `mere(jackie, X), mere(X, ling).`
(h) `pere(abraham, X), mere(mona, X).`
(i) `pere(X, marge), mere(Y, marge).`
(j) `pere(X, marge), pere(X, Z).`
(k) `pere(X, Y).`

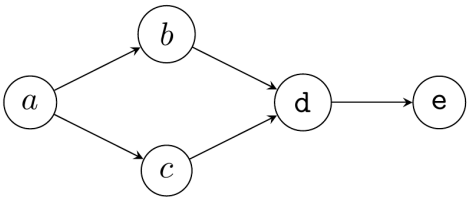
4. Quelle requête permet de trouver qui est la mère de la fille d'Abraham ?
5. Définir les prédicats suivants par des règles à ajouter dans le fichier puis les tester par des requêtes variées, en changeant les paramètres instanciés ou libres.

(a) `enfant(P, M, X)` qui est satisfait si `P` et `M` sont respectivement le père et la mère de `X`
(b) `grand_mere_paternelle(GM, X)`
(c) `parent(X, Y)` qui est satisfait si `X` est le père ou la mère de `Y`
(d) `grand_pere(GP, X)` qui est satisfait si `X` est un des grands-pères de `Y`
(e) `frere_ou_soeur(X, Y)`
(f) `tante(X, Y)`
(g) `cousin(X, Y)`
(h) `ancetre(X, Y)` qui est satisfait si `X` est un ancêtre de `Y` : l'un de ses parents, grands-parents, arrière-grands-parents, arrière-arrière-grand-parent, etc

Ainsi, on doit trouver 8 ancêtre à `bart` avec l'arbre généalogique fourni.

Exercice 15 – Graphe

On considère le graphe orienté sans cycle suivant :



1. En utilisant le prédicat binaire `arc`, représenter ce graphe par des faits en prolog.
2. Quelle requête permet d'afficher tous les nœuds accessibles depuis `a` ?
3. Quelle requête permet d'afficher tous les nœuds qui pointent vers `d` ?
4. Définir le prédicat `entre(X, S, D)` qui est satisfait si `X` est situé immédiatement entre `S` et `D`.

Par exemple, la requête `entre(X, a, d)` doit être satisfaite par `b` et `c`.
Par contre, les requêtes `entre(a, S, D)` `entre(X, a, b)` et `entre(X, a, e)` doivent échouer.

5. Définir le prédicat `chemin3(X, Y)` qui est satisfait si on peut aller de `X` à `Y` en au plus trois étapes.

Par exemple, la requête `chemin3(a, Y)` doit être satisfaite par `Y = b, c, d` ou `e`, la requête `chemin3(X, b)` doit être satisfaite uniquement par `X = a`.

6. Définir le prédicat `chemin(X, Y)` qui est satisfait si on peut aller de `X` à `Y`.
7. Que se passe-t-il si on ajoute un arc entre b et a ?